

Reading TypeScript: Functions, Async, Symbols

Three decoders for code you have never seen — annotated example, then every variation

October 2026

1. Reading a function

The shape

```
export async function getUser(id: string, retries = 3, opts?: Opts): Promise<User | null> {  
  .....  
  export async keyword name required default optional return type (Promise: async)  
  .....  
}
```

Find, in this order: the name; the parameters and which are optional; the declared return type; then every `return` statement in the body and ask whether each one matches the declared type.

Every function shape

Written	What it is	Notes
<code>function f(x: number): string { ... }</code>	declaration	hoisted; <code>this</code> depends on caller
<code>const f = (x: number): string => { ... }</code>	arrow assigned to a const	captures surrounding <code>this</code> ; braces need <code>return</code>
<code>const f = (x: number) => x * 2</code>	arrow, expression body	returns <code>x * 2</code> implicitly
<code>const f = function (x: number) { ... }</code>	function expression	rare in modern code
<code>async function f(): Promise<T></code>	async declaration	always returns a Promise, even with <code>return 5</code>
<code>const f = async (x) => { ... }</code>	async arrow	same
<code>f(x: number) { ... }</code> inside <code>class</code>	method	<code>this</code> is the instance when called as <code>obj.f()</code>
<code>static f() { ... }</code>	static method	called on the class: <code>A.f()</code>
<code>get total() { ... }</code>	getter	read as <code>obj.total</code> , no parentheses
<code>function f<T>(x: T): T</code>	generic	<code>T</code> is a placeholder for whatever is passed
<code>function f(x: string): x is Id</code>	type guard	returns boolean, narrows <code>x</code> for the caller
<code>function f(x: string): never</code>	never returns	throws or loops forever
<code>function f(): void</code>	returns nothing useful	a <code>return value</code> inside is a type error
<code>(x: number) => string</code> as a TYPE	function type	describes a callback parameter
<code>arr.map((x) => ...)</code>	callback	the arrow is handed to <code>map</code> , called once per element
<code>function f(a: string): void; function f(a: number): void; function f(a: any) {}</code>	overloads	the last signature is the implementation

Parameters

Written	Reads as	Inside the body
<code>x: number</code>	required	<code>x</code> is <code>number</code>
<code>x?: number</code>	optional	<code>x</code> is <code>number undefined</code> ; check before use
<code>x = 5</code>	default	<code>x</code> is <code>number</code> ; default applies only when the argument is <code>undefined</code> , not <code>null</code>
<code>x: number = 5</code>	default with explicit type	same
<code>...rest: number[]</code>	collects extra arguments	<code>rest</code> is always an array
<code>{ a, b }: Opts</code>	destructured object parameter	<code>a</code> and <code>b</code> are locals; the caller passes <code>{ a: 1, b: 2 }</code>
<code>{ a = 1, b }: Opts = {}</code>	destructured with defaults and a default object	caller may pass nothing

Written	Reads as	Inside the body
<code>[x, y]: [number, number]</code>	destructured tuple	
<code>this: Window</code> as first param	declares the type of <code>this</code>	not a real argument

What does it actually return? Trace every path

```
function find(xs: User[], id: string): User { // declares: always a User
  for (const u of xs) {
    if (u.id === id) return u; // path 1: a User ✓
  } // path 2: falls off the end → undefined x lies
  to the caller
}
```

Pattern	Returns
no <code>return</code> at all	<code>undefined</code>
<code>return;</code>	<code>undefined</code>
<code>if (...) return a;</code> with no else	<code>a</code> on that branch, <code>undefined</code> on the other
arrow with braces and no <code>return</code>	<code>undefined</code>
arrow with expression body	that expression
<code>async</code> function returning 5	<code>Promise<number></code> , resolves to 5
<code>async</code> function that throws	a rejected Promise, not a thrown error
<code>return await p</code> vs <code>return p</code>	same value; only <code>return await</code> is caught by a <code>try</code> in this function
<code>forEach(cb)</code>	always <code>undefined</code> , whatever <code>cb</code> returns
<code>arr.map(x => { x * 2 })</code>	array of <code>undefined</code> (braces, no return)

Calling it: trace one call

- Write down the arguments: `getUser("42")` → `id = "42"`, `retries = 3`, `opts = undefined`.
- Walk the body line by line with those values; write what each variable holds after the line.
- At each `if`, decide which branch with your values, and also note what the *other* branch would need.
- At each `return`, compare the value with the declared type.
- Repeat with an edge input: `""`, `undefined`, `0`, an id that does not exist.

Review questions for any function

- Does every `return` path match the declared return type? (missing path = `undefined`)
- Can any parameter be `undefined` where the body assumes it is not?
- Is every parameter with a `?` or a `= default` handled when the caller omits it?
- Does it mutate a parameter (`xs.sort()`, `opts.x = ...`) that the caller still uses?
- If `async`: is every call to it awaited by its callers?

2. Reading an async block

The annotated example

```
async function load(ids: string[]): Promise<User[]> {
  const cache = getCache();           // 1. sync; value right away
  const token = await getToken();      // 2. WAITS here; token is a string, not a Promise
  const p = fetchUser(ids[0]);         // 3. NO await: p is Promise<User>; request started
  const u = await p;                  // 4. now waits; u is a User

  const all = await Promise.all(      // 5. starts every fetch at once, waits for all
    ids.map((id) => fetchUser(id))    //   the map returns Promise<User>[]
  );                                  //   all is User[]

  ids.forEach(async (id) => {         // 6. x forEach ignores the returned Promise;
    await save(id);                  //   these run, but load() does not wait
  });

  try {
    const r = fetchUser("x");         // 7. x not awaited: rejection escapes this try
    return [u, ...all];
  } catch (e) {                       // 8. e is unknown; narrow before e.message
    throw new Error("load failed", { cause: e });
  }
}
```

Promise or value? Decide for every line

Line looks like	The variable holds	Is this a bug?
<code>const x = await f()</code>	the resolved value	✓
<code>const x = f()</code> where <code>f</code> is async	a <code>Promise</code> ; <code>x.name</code> is <code>undefined</code>	✗ unless <code>x</code> is awaited later or passed to <code>Promise.all</code>
<code>const x = f().then(g)</code>	a <code>Promise</code> of <code>g</code> 's result	✓ if awaited or returned
<code>return f()</code> inside async	the <code>Promise</code> , flattened for the caller	✓, but not caught by this function's <code>try</code>
<code>return await f()</code>	same value	✓, and caught by this function's <code>try</code>
<code>f();</code> alone on a line, <code>f</code> async	fire-and-forget	✗ unless <code>.catch</code> is attached or intended
<code>await f(); await g();</code>	sequential	✓ if <code>g</code> needs <code>f</code> 's result; slow if independent
<code>await Promise.all([f(), g()])</code>	parallel, array of results in order	✓; rejects on the FIRST failure
<code>await Promise.allSettled([...])</code>	array of <code>{status, value reason}</code>	✓ when partial failure is acceptable
<code>arr.map(async x => ...)</code>	an array of <code>Promises</code>	✗ until wrapped in <code>Promise.all</code>
<code>arr.forEach(async x => ...)</code>	nothing; callbacks run detached	✗ almost always
<code>for (const x of arr) await f(x)</code>	sequential, one at a time	✓ if order matters; otherwise slow
<code>await</code> inside a non-async function	syntax error	
<code>res.json()</code>	a <code>Promise</code>	✗ if used without <code>await</code>
<code>JSON.parse(s)</code>	a value (sync)	throws on bad input; needs <code>try</code>

Where does an error go?

Situation	Where the error lands
<code>throw</code> inside an async function	the returned <code>Promise</code> rejects
<code>await p</code> where <code>p</code> rejects	thrown at the <code>await</code> line; caught by an enclosing <code>try</code>
<code>p</code> not awaited, rejects	unhandled rejection; crashes Node or logs in browser
<code>try { f() } catch {}</code> with async <code>f</code>	NOT caught; the <code>Promise</code> escapes the <code>try</code>
<code>try { await f() } catch {}</code>	caught
<code>try { return f() } catch {}</code>	NOT caught
<code>try { return await f() } catch {}</code>	caught
<code>Promise.all</code> with one rejection	rejects immediately with that one error; others keep running but are ignored
<code>.catch(() => {})</code>	swallowed silently; a finding unless intended

Situation	Where the error lands
<code>catch (e) { e.message }</code>	type error: <code>e</code> is <code>unknown</code> ; needs <code>instanceof Error</code>

Shared state across an `await`

Every `await` is a point where another request can run. Read-then-await-then-write on anything shared (a `Map`, a module-level variable, a counter) is a race.

```
const n = counter.get(k) ?? 0;    // read
await db.save(...);              // another request runs here and reads the same n
counter.set(k, n + 1);           // write: one increment lost
```

Cache version: `if (!cache.has(k)) cache.set(k, await load(k))` lets every concurrent miss call `load`. Fix: store the Promise, `cache.set(k, load(k))`, then `await cache.get(k)`.

Timers

Written	Meaning	Trap
<code>setTimeout(fn, ms)</code>	run <code>fn</code> once after <code>ms</code>	returns an id; <code>clearTimeout(id)</code> cancels
<code>setInterval(fn, ms)</code>	run <code>fn</code> every <code>ms</code>	never cleared = leak; <code>.unref()</code> lets Node exit
<code>await new Promise(r => setTimeout(r, ms))</code>	sleep	the Python <code>await asyncio.sleep</code>
<code>setTimeout(fn, 0)</code>	run after all pending microtasks	still after every <code>.then</code>
<code>Date.now()</code>	ms since epoch, a number	hard-coded = untestable; inject a clock

Review questions for any async block

- For each line: Promise or value? Is every Promise awaited, returned, or passed to `Promise.all`?
- Is there an `async` callback inside `forEach`?
- Is there a `try` whose body has an un-awaited call?
- Independent awaits in sequence that could be parallel?
- Shared state read before an `await` and written after?
- `res.ok` checked before `res.json()`?
- Interval cleared or `.unref()`'d?

3. Symbol decoder

The same character means different things by position. Find the position first, then the meaning.

?

Position	Meaning	Example
after a property name in a type	optional property	<code>{ a?: number }</code>
after a parameter name	optional parameter	<code>f(x?: string)</code>
between two expressions with <code>:</code>	ternary (if-else)	<code>ok ? a : b</code>
before <code>.</code> or <code>[</code> or <code>(</code>	optional chaining	<code>o?.a</code> , <code>arr?.[0]</code> , <code>cb?.()</code>
<code>??</code>	nullish fallback	<code>x ?? 10</code>
<code>??=</code>	assign if nullish	<code>x ??= 10</code>

!

Position	Meaning	Example
before an expression	logical NOT	<code>!x</code> , <code>!arr.length</code>
after an expression	non-null assertion (compile-time only)	<code>user!.name</code>
<code>!==</code>	strict not-equal	<code>a !== b</code>
<code>!=</code>	loose not-equal (flag it)	<code>a != b</code>

:

Position	Meaning	Example
after a name in a declaration or parameter	type annotation	<code>x: number</code>
after <code>)</code> of a function	return type	<code>f(): string</code>
inside <code>{ }</code> in a VALUE	object key-value	<code>{ a: 1 }</code>
inside <code>{ }</code> in a TYPE	property type	<code>{ a: number }</code>
in destructuring	rename	<code>const { a: x } = o</code>
second part of a ternary	else	<code>c ? a : b</code>
after <code>case</code> / label	switch case	<code>case "x":</code>

=, ==, ===, =>

Written	Meaning		
<code>=</code>	assignment (never comparison)		
<code>==</code>	loose equality with coercion (flag)		
<code>===</code>	strict equality		
<code>=></code>	arrow function: "returns"		
<code>>=</code> , <code><=</code>	comparison; check the boundary value		
<code>+=</code> , <code>-=</code> , <code>*=</code>	compound assignment		
<code>`</code>		<code>=</code> , <code>&&=</code> , <code>??=</code>	assign if falsy / truthy / nullish

. and ...

Written	Meaning	Example
<code>.</code>	property or method access, chains left to right	<code>a.b.c()</code> , <code>arr.map(f).filter(g)</code>
<code>?.</code>	optional access	<code>a?.b</code>
<code>...x</code> in <code>[]</code> or <code>{ }</code>	spread: copy elements/keys in	<code>[...a, 1]</code> , <code>{ ...o, k: v }</code>
<code>...x</code> in a parameter list	rest: collect the remaining arguments	<code>(a, ...rest)</code>
<code>...x</code> in destructuring	rest: the remaining keys/elements	<code>const { a, ...rest } = o</code>

Brackets

Written	Meaning	Example
<code>{ }</code> in a value	object literal	<code>{ a: 1 }</code>
<code>{ }</code> after <code>=></code> , <code>if</code> , <code>for</code> , <code>function</code>	block body	<code>() => { return 1 }</code>
<code>{ }</code> in a type	object type	<code>{ a: number }</code>
<code>{ }</code> in <code>import</code>	named imports	<code>import { x } from "m"</code>
<code>{ }</code> in destructuring	pull keys out	<code>const { a } = o</code>
<code>[]</code> in a value	array literal	<code>[1, 2]</code>
<code>[]</code> after a name	index access	<code>arr[0], obj["k"]</code>
<code>[]</code> after a type	array of	<code>number[]</code>
<code>[]</code> in a type with commas	tuple	<code>[string, number]</code>
<code>[]</code> in destructuring	pull elements out	<code>const [a, b] = arr</code>
<code>[k]: v</code> in an object literal	computed key	<code>{ [name]: 1 }</code>
<code>()</code> after a name	call	<code>f(x)</code>
<code>()</code> around params	parameter list	<code>(a, b) => ...</code>
<code>()</code> around <code>{ }</code> after <code>=></code>	return an object literal	<code>() => ({ a: 1 })</code>
<code>< ></code> after a name	generic argument	<code>Map<string, number>, Promise<User></code>
<code>< ></code> before <code>(</code> in a declaration	generic parameter	<code>function f<T>(x: T)</code>
<code>`</code>	template string	<code>`Hello \${name}`</code>
<code>\${ }</code> inside backticks	interpolation	<code>`\${a + b}`</code>

|, &, ||, &&

Written	Where	Meaning
<code> </code>	in a type	union: either
<code>&</code>	in a type	intersection: both
<code> </code>	in a value	OR; fallback on falsy
<code>&&</code>	in a value	AND; <code>a && b</code> is <code>b</code> if <code>a</code> truthy else <code>a</code>
<code> </code> , <code>&</code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code>	in a value with numbers	bitwise (rare; if seen, read as integer math)

Keywords that look like operators

Written	Meaning	Runtime effect
<code>as T</code>	cast	none
<code>as const</code>	literal types	none
<code>satisfies T</code>	check without widening	none
<code>typeof x</code>	runtime type name as a string	yes
<code>typeof x</code> in a type position	the type of <code>x</code>	none
<code>instanceof C</code>	is <code>x</code> built from class <code>C</code>	yes
<code>in</code>	<code>"k" in obj</code> key check / <code>for...in</code> keys	yes
<code>of</code>	<code>for...of</code> values	yes
<code>keyof T</code>	union of key names	none
<code>new C()</code>	construct an instance	yes
<code>await p</code>	wait and unwrap	yes
<code>yield</code>	generator step	yes
<code>delete o.k</code>	remove a key	yes
<code>void expr</code>	evaluate and return <code>undefined</code>	yes
<code>declare</code>	exists elsewhere	none
<code>readonly</code>	cannot reassign (type only)	none
<code>#x</code>	runtime-private class field	yes
<code>@decorator</code>	wraps a class/method	yes

Three-second decode

1. Is this line a type (after `:` or inside `interface/type`) or a value? Types run nothing.
2. Find `=`: left is what is being declared, right is the value.
3. Find `=>`: left is parameters, right is what is returned.
4. Find `await`: the thing after it is a Promise being unwrapped; a Promise-returning call without `await` is the bug to look for.
5. Find `?`, `!`, `as`: each is a place the author claimed something about nullness or type; ask whether the claim is true at runtime.